

Cahier des charges — Plateforme e-commerce solaire inspirée de Wattuneed

Présentation du projet

Le projet consiste à concevoir et développer une plateforme e-commerce spécialisée dans les kits solaires, batteries, onduleurs, accessoires et services associés. Le site cible doit s'inspirer du positionnement fonctionnel de la page de référence Wattuneed, qui met en avant un catalogue de kits solaires, des filtres techniques, des guides d'aide au choix et une FAQ orientée expertise photovoltaïque [cite:16][cite:17].

L'objectif n'est pas de reproduire l'interface existante à l'identique, mais de créer une version plus moderne, plus claire, plus performante et plus scalable, avec une stack technique basée sur React pour le frontend et Java Spring Boot pour le backend. La nouvelle plateforme devra améliorer l'expérience utilisateur, la conversion commerciale, l'administration du catalogue et l'évolutivité technique.

Objectifs métier

La plateforme devra permettre la vente en ligne de kits solaires selon plusieurs usages : autoconsommation, site isolé, plug and play, mobilité, stockage batterie et équipements complémentaires. Le site de référence présente déjà plusieurs familles de produits et des critères de sélection techniques avancés, ce qui confirme la nécessité d'un modèle métier riche et d'un tunnel de décision guidé [cite:17].

Les objectifs métier sont les suivants :

- Vendre des kits solaires et accessoires à des particuliers et professionnels.
- Générer des demandes de devis qualifiées.
- Faciliter le choix du bon kit grâce à un assistant et un simulateur.
- Valoriser le contenu SEO via guides, FAQ et pages catégories.
- Permettre une gestion avancée du catalogue, des commandes et des contenus.
- Préparer une évolution future vers le multilingue, multi-pays et B2B.

Périmètre fonctionnel

La plateforme sera découpée en trois grands périmètres : front-office client, back-office administrateur et services métier transverses. Cette approche est cohérente avec le site de référence, qui combine catalogue, contenu éditorial, aide à la décision et logique commerciale [cite:17].

Front-office client

Le front-office devra inclure les modules suivants :

- Page d'accueil orientée conversion.
- Catalogue produits avec filtres avancés.
- Pages catégories et sous-catégories.
- Pages marques.
- Fiche produit détaillée.
- Comparateur de produits.
- Simulateur solaire.
- Assistant de choix de kit.
- Blog et centre d'aide.
- Panier.
- Tunnel de commande.
- Espace client.
- Système de favoris.
- Suivi de commande.
- Formulaire de devis.
- Contact, SAV et assistance.

Back-office administrateur

Le back-office devra permettre :

- Gestion des produits et variantes.
- Gestion des catégories et attributs.
- Gestion des stocks.
- Gestion des prix, promotions et coupons.
- Gestion des commandes.
- Gestion des clients.
- Gestion des demandes de devis.
- Gestion du contenu CMS.
- Gestion des articles de blog et FAQ.
- Paramétrage SEO.
- Gestion des rôles et permissions.
- Tableau de bord commercial et technique.
- Journalisation des actions administratives.

Services métier transverses

Les services métier devront couvrir :

- Moteur de filtrage catalogue.
- Moteur de recherche.
- Calcul de disponibilité stock.
- Recommandation de kits.
- Estimation de production solaire.
- Calcul des taxes.
- Calcul des frais de livraison.
- Notifications email.
- Import/export de catalogue.
- Audit et journalisation.

Benchmark fonctionnel

La page de référence présente un catalogue de kits solaires avec filtrage par usage, nombre de panneaux, phase, batterie, type d'injection, tension et prix, ainsi qu'un contenu d'accompagnement pour aider l'utilisateur à choisir rapidement sa solution [cite:17]. Elle expose aussi une logique éditoriale avec FAQ et contenus techniques, utile pour le référencement et la réassurance avant achat [cite:17].

Le nouveau site devra reprendre les points forts suivants :

- Richesse du catalogue.
- Profondeur des filtres.
- Aide au choix rapide.
- Mise en avant du conseil expert.
- Couverture de plusieurs types d'installation.

Les améliorations attendues sont les suivantes :

- Design premium et plus respirant.
- Navigation plus claire.
- Comparaison produits plus efficace.
- Fiches produit mieux hiérarchisées.
- Parcours devis plus fluide.
- Mobile UX plus aboutie.
- Architecture frontend et backend plus maintenable.
- Meilleure vitesse perçue et meilleure lisibilité.

Utilisateurs cibles

Les utilisateurs cibles sont :

- Particuliers recherchant un kit solaire pour maison, balcon ou site isolé.
- Professionnels souhaitant équiper des bâtiments, bureaux, exploitations ou locaux techniques.
- Utilisateurs peu techniques nécessitant un assistant de choix.
- Utilisateurs avancés souhaitant comparer des données techniques précises.
- Administrateurs internes gérant catalogue, commandes, contenus et demandes commerciales.

Parcours utilisateurs principaux

Parcours 1 — Achat direct

1. L'utilisateur arrive sur la page d'accueil ou une page catégorie.
2. Il filtre les produits selon son besoin.
3. Il consulte une fiche produit.
4. Il compare éventuellement plusieurs kits.
5. Il ajoute un produit au panier.
6. Il passe commande.
7. Il suit sa commande depuis son espace client.

Parcours 2 — Besoin d'accompagnement

1. L'utilisateur lance l'assistant de choix ou le simulateur.
2. Il saisit ses besoins : type d'installation, consommation, zone, objectifs.
3. Le système recommande une ou plusieurs familles de kits.
4. L'utilisateur peut demander un devis.
5. L'équipe commerciale reprend la demande depuis le back-office.

Parcours 3 — Découverte SEO

1. L'utilisateur arrive depuis Google sur un guide, une FAQ ou une page catégorie.
2. Il consomme le contenu pédagogique.
3. Il navigue vers les produits associés.
4. Il lance une simulation ou remplit un formulaire de devis.

Architecture frontend

Le frontend sera développé avec React, dans une logique de composants réutilisables, d'optimisation des performances et de séparation claire entre composants de présentation et logique métier.

Stack frontend recommandée

Domaine	Choix recommandé
Framework UI	React
Bundler	Vite
Routing	React Router
State management	Redux Toolkit ou Zustand
Data fetching	React Query
Formulaires	React Hook Form + Zod
UI system	Tailwind CSS ou Material UI customisé
Internationalisation	react-i18next
Tests unitaires	Vitest + React Testing Library
Tests end-to-end	Playwright

Arborescence frontend suggérée

```
frontend/  
├── src/  
│   ├── app/  
│   ├── components/  
│   ├── features/  
│   │   ├── auth/  
│   │   ├── catalog/  
│   │   ├── cart/  
│   │   ├── checkout/  
│   │   ├── quotes/  
│   │   ├── estimator/  
│   │   ├── account/  
│   │   └── cms/  
│   ├── hooks/  
│   ├── layouts/  
│   ├── pages/  
│   ├── routes/  
│   ├── services/  
│   ├── store/  
│   ├── styles/  
│   ├── types/  
│   └── utils/  
└── public/
```

Pages frontend à développer

Page	Objectif
Accueil	Présenter l'offre, rassurer et orienter
Listing catégorie	Explorer, filtrer et trier les produits
Fiche produit	Détail technique, prix, variantes, CTA
Comparateur	Comparer plusieurs produits sur les critères clés
Simulateur	Recommander une solution selon le besoin
Panier	Préparer la commande
Checkout	Finaliser achat, livraison et paiement
Compte client	Historique, profil, devis, favoris
Blog/FAQ	Contenu SEO et aide au choix
Devis	Collecte du besoin commercial
Contact/SAV	Support avant et après vente

Composants frontend principaux

- Header sticky.
- Mega menu catégories.
- Barre de recherche avec autocomplétion.
- ProductCard.
- ProductGallery.
- ProductFiltersPanel.
- CompareBar.
- ProductSpecsTable.
- ProductBadges.
- EstimatorWizard.
- QuoteForm.
- CartSummary.
- CheckoutStepper.
- ReviewList.
- SeoContentSection.
- CmsBlockRenderer.
- Breadcrumbs.
- Pagination.
- Toast système.

- Skeleton loaders.

Exigences UX frontend

- Mobile first.
- Temps de chargement perçu optimisé.
- Filtres visibles sur desktop, drawer sur mobile.
- Recherche globale accessible depuis toutes les pages.
- CTA principaux visibles mais non agressifs.
- Parcours d'achat simple en moins de 4 étapes.
- Accès rapide au devis depuis les pages stratégiques.
- Accessibilité WCAG AA.
- Support du multilingue dès l'architecture initiale.

Design system et direction artistique

Le design devra transmettre la confiance, la technicité et la transition énergétique. La référence actuelle met l'accent sur l'expertise produit et le volume d'offre, mais la nouvelle interface devra être plus structurée visuellement et plus premium dans son rendu [cite:17].

Positionnement visuel

- Style : clean tech premium.
- Ton : professionnel, rassurant, moderne.
- Ambiance : énergie propre, sobriété, performance.
- Orientation UX : clarté, comparabilité, pédagogie.

Palette couleur recommandée

Usage	Couleur	Hex
Primary	Deep Teal	#0F766E
Primary hover	Dark Teal	#115E59
Secondary	Solar Gold	#F59E0B
Success	Green	#22C55E
Error	Rose Red	#E11D48
Background	Light Slate	#F8FAFC
Surface	White	#FFFFFF
Surface alt	Soft Gray	#F1F5F9
Border	Slate 300	#CBD5E1

Usage	Couleur	Hex
Text primary	Slate 900	#0F172A
Text secondary	Slate 600	#475569

Typographie

- Police principale : Inter ou General Sans.
- Police de renfort possible pour titres marketing : Satoshi ou Cabinet Grotesk.
- Corps de texte entre 16px et 18px.
- Hiérarchie claire avec 4 niveaux maximum par page.

Règles UI

- Grille responsive 12 colonnes desktop, 6 tablette, 4 mobile.
- Radius de 10 à 14px selon composants.
- Bordures légères et ombres discrètes.
- Icônes simples, linéaires, cohérentes.
- Très peu de couleurs saturées dans une même vue.
- CTA principaux en teal, accents informatifs en gold.

Améliorations visuelles attendues

- Hero plus impactant et plus lisible.
- Mise en avant plus nette des catégories.
- Fiches produit avec sections repliables.
- Tableaux techniques mieux stylés.
- Comparateur produit moderne.
- Assistant de choix sous forme de wizard.
- Cartes produit plus claires avec badges normalisés.
- Zone de réassurance plus professionnelle.

Architecture backend

Le backend sera développé en Java 21 avec Spring Boot 3 selon une architecture modulaire claire. Il devra supporter la complexité d'un catalogue technique, la gestion des commandes, les devis et les contenus. Cette approche est justifiée par la diversité des familles de produits et critères techniques déjà visibles sur la plateforme de référence [cite:17].

Stack backend recommandée

Domaine	Choix recommandé
Langage	Java 21
Framework	Spring Boot 3
API	REST JSON
Sécurité	Spring Security + JWT + Refresh Token
Base de données	PostgreSQL
Cache	Redis
Recherche	PostgreSQL Full Text puis Elasticsearch si besoin
Fichiers	MinIO ou Amazon S3
Documentation API	OpenAPI / Swagger
Paieement	Stripe, PayPal ou fournisseur local
Emails	SMTP ou service transactionnel
Build	Maven
Containerisation	Docker

Architecture logique backend

```
backend/  
├─ src/main/java/com/company/solar/  
│   ├── config/  
│   ├── security/  
│   ├── common/  
│   ├── auth/  
│   ├── user/  
│   ├── catalog/  
│   ├── category/  
│   ├── brand/  
│   ├── product/  
│   ├── pricing/  
│   ├── stock/  
│   ├── cart/  
│   ├── order/  
│   ├── payment/  
│   ├── shipment/  
│   ├── quote/  
│   ├── estimator/  
│   ├── review/  
│   ├── cms/  
│   ├── blog/  
│   ├── faq/  
│   ├── seo/  
│   └─ notification/
```

Modules backend

- Authentification et autorisation.
- Gestion utilisateurs.
- Catalogue et catégories.
- Produits, variantes et attributs.
- Prix et promotions.
- Stock et disponibilité.
- Panier.
- Commandes.
- Paiement.
- Livraison.
- Demandes de devis.
- Simulateur solaire.
- CMS.
- Blog et FAQ.
- SEO.
- Notifications.
- Audit et monitoring.

Endpoints API cibles

Endpoint	Usage
/api/auth/*	Login, register, refresh token
/api/users/me	Données utilisateur connecté
/api/categories	Liste et arborescence catégories
/api/products	Recherche et listing produits
/api/products/{slug}	Détail produit
/api/filters	Filtres de catalogue
/api/compare	Comparaison produits
/api/cart	Gestion panier
/api/checkout	Préparation checkout
/api/orders	Commandes client

Endpoint	Usage
/api/quotes	Demandes de devis
/api/estimator	Simulation et recommandation
/api/blog	Articles
/api/faq	FAQ
/api/admin/*	Administration

Base de données

Le schéma de base de données doit être pensé pour un catalogue technique fortement filtrable. La plateforme de référence distingue plusieurs types de kits, puissances, options batterie et critères de configuration, ce qui impose un modèle flexible fondé sur produits, variantes et attributs [cite:17].

Tables principales

Table	Rôle
users	Comptes clients et administrateurs
roles	Rôles sécurité
user_roles	Association utilisateurs-rôles
addresses	Adresses facturation et livraison
categories	Arborescence catalogue
brands	Marques
products	Produits principaux
product_variants	Variantes et déclinaisons
product_images	Visuels produits
product_documents	Fiches PDF et documents techniques
attributes	Définition des attributs
attribute_values	Valeurs possibles des attributs
product_attribute_values	Liaison produit-attribut
prices	Prix courants et historiques
stock_items	Gestion stock
carts	Paniers
cart_items	Lignes panier
orders	Commandes
order_items	Lignes commandes

Table	Rôle
payments	Transactions de paiement
shipments	Expéditions
quotes	Demandes de devis
quote_items	Produits liés aux devis
reviews	Avis clients
blog_posts	Articles SEO
faq_items	FAQ
coupons	Réductions
seo_pages	Métadonnées SEO
audit_logs	Journal d'audit

Structure détaillée de la table `products`

Champ	Type	Description
id	bigint	Identifiant technique
sku	varchar	Référence interne
slug	varchar	URL SEO
name	varchar	Nom produit
short_description	text	Description courte
long_description	text	Description longue
category_id	bigint	Catégorie
brand_id	bigint	Marque
product_type	varchar	Kit, batterie, onduleur, accessoire
installation_type	varchar	Autoconsommation, isolé, plug and play
phase_type	varchar	Monophasé, triphasé
base_power_kwc	numeric	Puissance kit
inverter_power_va	numeric	Puissance onduleur
battery_capacity_kwh	numeric	Capacité batterie
voltage_output	varchar	Tension de sortie
injection_type	varchar	Réinjection ou non
is_active	boolean	Statut publication
created_at	timestamp	Création
updated_at	timestamp	Mise à jour

Structure détaillée de la table `product_variants`

Champ	Type	Description
id	bigint	Identifiant
product_id	bigint	Produit parent
reference	varchar	Référence commerciale
label	varchar	Libellé de variante
price_ht	numeric	Prix HT
price_ttc	numeric	Prix TTC
currency	varchar	Devise
stock_quantity	integer	Quantité en stock
weight	numeric	Poids
dimensions	varchar	Dimensions
is_default	boolean	Variante par défaut
is_active	boolean	Statut

Structure détaillée de la table `attributes`

Champ	Type	Description
id	bigint	Identifiant
code	varchar	Code technique
name	varchar	Nom affiché
type	varchar	text, number, boolean, select, range
filterable	boolean	Utilisable dans les filtres
comparable	boolean	Utilisable dans le comparateur
unit	varchar	Unité

Relations clés

- Une catégorie possède plusieurs produits.
- Un produit peut avoir plusieurs variantes.
- Un produit peut être lié à plusieurs attributs.
- Un produit peut contenir plusieurs images et documents.
- Un utilisateur peut avoir plusieurs commandes et devis.
- Un devis peut être converti en commande.
- Une commande contient plusieurs lignes de commande.

Sécurité

Authentification

- JWT access token.
- Refresh token sécurisé.
- Gestion des rôles : client, commercial, admin, super admin.
- Mot de passe hashé avec BCrypt.
- Protection contre brute force.
- Réinitialisation mot de passe par email.

Sécurité applicative

- Validation stricte des entrées.
- Protection CSRF selon mode d'auth choisi.
- Rate limiting sur endpoints sensibles.
- Journalisation des actions critiques.
- Gestion centralisée des exceptions.
- Contrôle d'accès par ressource et rôle.
- Upload fichiers sécurisé.

SEO et contenu

Le site devra capitaliser sur des pages catégories riches, des FAQ et des contenus pédagogiques, à l'image de la logique éditoriale observée sur le site de référence [cite:17]. Le SEO devra être intégré nativement au modèle produit et CMS.

Exigences SEO

- URLs propres et lisibles.
- Balises title et meta description administrables.
- Données structurées produits et FAQ.
- Breadcrumbs.
- Sitemap XML.
- Robots.txt configurable.
- Pages catégories enrichies.
- Blog orienté guides pratiques.
- Pages marques indexables.
- Gestion canonicals.
- Open Graph et Twitter Cards.

Performance

Frontend

- Lazy loading images.
- Code splitting par route.
- Mise en cache HTTP.
- Skeletons pendant chargement.
- Optimisation des bundles.
- Préchargement des routes à forte probabilité.

Backend

- Cache Redis sur catégories, filtres et fiches fréquentes.
- Pagination systématique.
- Index SQL sur colonnes de recherche.
- Compression GZIP.
- Logs centralisés.
- Monitoring applicatif.

Intégrations externes

- Paiement en ligne.
- Email transactionnel.
- Service de transport.
- Tracking analytics.
- Recaptcha ou solution anti-spam.
- Hébergement fichiers produit.
- Outils CRM possibles en phase 2.

Back-office détaillé

Dashboard administrateur

Le dashboard devra afficher :

- Chiffre d'affaires.
- Nombre de commandes.
- Panier moyen.
- Produits les plus vus.
- Produits les plus vendus.

- Demandes de devis récentes.
- État du stock critique.
- Performance des catégories.

Gestion catalogue

- CRUD catégories.
- CRUD produits.
- CRUD variantes.
- CRUD attributs.
- Gestion médias.
- Import CSV/Excel.
- Prévisualisation avant publication.
- Gestion badges marketing.

Gestion commerciale

- Pipeline devis.
- Statuts commande.
- Facturation exportable.
- Filtres clients.
- Historique d'actions.
- Notes internes.

Règles métier principales

- Un produit inactif ne doit pas apparaître en front.
- Un produit hors stock peut rester visible mais non commandable selon paramétrage.
- Une demande de devis peut être créée depuis plusieurs points d'entrée.
- Les filtres ne doivent afficher que les valeurs disponibles dans le contexte courant.
- Le comparateur doit supporter au moins 4 produits.
- Un contenu CMS doit être publiable et dépubliable sans déploiement.
- Une commande doit conserver le snapshot du produit acheté.

Tests et qualité

Frontend

- Tests unitaires composants.
- Tests intégration formulaires.
- Tests E2E parcours catalogue, panier, checkout et devis.

Backend

- Tests unitaires services.
- Tests repository.
- Tests API.
- Tests sécurité.
- Tests de charge sur listing catalogue et checkout.

Qualité globale

- Convention de code.
- Revue de code obligatoire.
- SonarQube possible.
- CI/CD avec build, test et déploiement contrôlé.

Déploiement et infrastructure

Environnements

- Local.
- Dev.
- Staging.
- Production.

Infrastructure recommandée

- Frontend déployé sur CDN ou serveur Nginx.
- Backend Spring Boot conteneurisé.
- PostgreSQL managé ou dédié.
- Redis pour cache.
- MinIO/S3 pour assets.
- Reverse proxy Nginx.
- Certificats SSL.
- Sauvegardes automatiques.
- Supervision et alerting.

Planning indicatif

Phase	Contenu
Phase 1	Cadrage fonctionnel et benchmark
Phase 2	UX, UI, prototype et design system
Phase 3	Conception BDD et API
Phase 4	Développement backend
Phase 5	Développement frontend
Phase 6	Recette, optimisation, SEO, sécurité
Phase 7	Mise en production

Critères d'acceptation

- Le catalogue est navigable et filtrable sans friction.
- Les fiches produit sont complètes et lisibles.
- Le comparateur fonctionne correctement.
- Le devis est accessible et traçable.
- Le paiement et le checkout sont stables.
- Le back-office couvre la gestion complète du catalogue et des commandes.
- Le SEO technique est en place.
- Les performances sont conformes aux objectifs.
- La sécurité et les rôles sont validés.

Livrables attendus

- Cahier des charges validé.
- Maquettes UI/UX.
- Design system.
- Schéma base de données.
- Documentation API.
- Code source frontend.
- Code source backend.
- Scripts de déploiement.
- Documentation d'exploitation.
- Plan de tests.

Recommandation de découpage MVP

MVP phase 1

- Accueil.
- Catalogue.
- Fiche produit.
- Panier.
- Checkout.
- Demande de devis.
- Espace client basique.
- Back-office produits, catégories, commandes, devis.

Phase 2

- Comparateur avancé.
- Simulateur solaire.
- Blog.
- Avis clients.
- Promotions avancées.
- Multilingue.
- Multi-pays.

Conclusion

Le projet doit reprendre la richesse fonctionnelle du modèle Wattuneed — catalogue technique, aide au choix, filtres spécialisés et contenus d'accompagnement — tout en proposant une expérience plus moderne, plus claire et plus évolutive [cite:16][cite:17]. Une architecture React + Spring Boot avec PostgreSQL, Redis et un back-office robuste constitue une base solide pour lancer une plateforme solaire e-commerce professionnelle et scalable.